

```
fd=-39.438293457031 -> 1.0011011100000011010000*2^5
$ GSL_IEEE_MODE="extended-precision" ./gsl_ieee
GSL_IEEE_MODE="extended-precision,trap-common"
fd=-84.018717155 -> 1.010100000001001100011000110010000000010011011*2^6
fd=-39.438293457031 -> 1.0011011100000011010000*2^5
```

Notice the change in different values (especially the first one), depending on precision and rounding off. From the above output, you can infer that the structure of the finite number, $(-1)^s \times c \times b^q$, has been observed while representing and storing the numbers. The base here is '2' (binary) and not '10' (decimal). Since most of us are used to decimal format, working manually with these numbers could be a bit tricky.

Debugging GSL programs using GDB

Debugging is a major part of programming that no one can escape. Most of us cannot write a totally correct program in the first attempt. GDB (GNU Debugger) actually stands apart from GSL, but they can be used together to find bugs in our programs. Most of us who are well acquainted with GCC might also know GDB. This section actually assumes that the basic commands are known to the reader. If not, you can go through one of the many tutorials available on the Internet.

The command-line interface is available, but I would rather recommend the user interface that even shows you your source code, and in which you can debug, step by step. I will re-use the program `gsl_ieee.c`, which we used in the IEEE floating-point-arithmetic segment. Let's take a look at the usual commands on the command-line interface.

Compiling:

```
$ gcc -Wall -g gsl_ieee.c -lgsl -lgslcblas -lm -o gsl_ieee
```

Debugging:

```
$ gdbtui gsl_ieee.c
```

As soon as you enter this command, you should see an interface like that shown in Figure 1. From here onwards, the command-line prompt will be `(gdb)` instead of `$`. The obvious steps would be to set breakpoints, and thereafter debug them. To set `GSL_IEEE_MODE` while using GDB, enter the following commands:

```
(gdb) info signal SIGFPE
Signal Stop Print Pass to program Description
SIGFPE Yes Yes Yes Arithmetic exception
(gdb) handle SIGFPE stop nopass
Signal Stop Print Pass to program Description
SIGFPE Yes Yes No Arithmetic exception
(gdb) set env GSL_IEEE_MODE=extended-precision
```

Thereafter, you can set the breakpoints, and run the program as usual.



Figure 1: GDB TUI

BLAS support

The Basic Linear Algebra Sub-programs (BLAS) contains functions for fundamental operations on vectors and matrices, which can be used to optimise vector-matrix operations. GSL requires a BLAS library (supplied along with the package) for vector and matrix operations. This can be replaced by the tuned ATLAS library, for better performance. While there are limitations to BLAS that could be overcome by ATLAS, the former can satisfy the needs of most users without modification. Unless you really need to change, I suggest you do not do so. BLAS is divided into different levels of sub-programs:

- Vector-vector (Level 1)
- Matrix-vector (Level 2)
- Matrix-matrix (Level 3) operations.

GSL for Windows

Well, I must say that GSL has not left Windows users unhappy. Mostly, using it on Windows is similar to how you use it on Linux. To download the installer and for instructions to install GSL on Windows, visit: <http://gnuwin32.sourceforge.net/packages/gsl.htm>

Well, the list of utilities is too long to be included here. To refer to the list of add-ons, you could visit the GSL homepage.

Winding up the series

Finally, concluding this series of articles on GSL, I hope that you had an exciting learning experience. I have tried to cover as much as possible, but there could be gaps. It was a great pleasure to be of help to those who contacted me. I would personally like to thank *LFY* for giving me such a wonderful opportunity to write for this popular magazine. 

By: Amol Takkalki

The author is a FOSS enthusiast, whose interests include data mining, neural networks, process automation and high-performance/parallel computing. His hobbies are programming, photography, Linux and music. You can contact him at takkalki.amol@gmail.com. He strongly believes that, "Open source rocks!"